

# Об'єктно-орієнтоване програмування

## Частина 2: інкапсуляція, перевантаження операторів

### Інкапсуляція

Над екземплярами (атрибутами екземплярів) можна виконувати різні операції. Наприклад, можна збільшити вік особи (ми вже це робили у першій частині):

```
class Person:
    info = "I'm a Person."

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"{self.name}, {self.age} years old."

alice = Person('Alice', 20)
print(alice)      # Alice, 20 years old

alice.age += 2
print(alice)      # Alice, 22 years old
```

Але на практиці атрибути екземплярів зазвичай не змінюють безпосередньо, а використовують інкапсуляцію — концепцію проєктування, основна ідея якої полягає в реалізації операцій над об'єктами у вигляді методів класу. У нашому випадку доцільно додати до класу `Person` метод `grew_up()`, який збільшує вік особи<sup>1</sup>:

```
class Person:
    info = "I'm a Person."

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"{self.name}, {self.age} years old."
```

---

<sup>1</sup> Насправді, цей метод може і зменшити вік, якщо йому передати від'ємне число. Але будемо вважати без додаткових перевірок, що методу передаються додатні числа.

```
def grew_up(self, years=1):  
    self.age += years
```

Тоді ця операція здійснюватиметься не безпосередніми перетвореннями, а викликом методу `grew_up()`:

```
alice = Person('Alice', 20)  
print(alice)          # Alice, 20 years old  
  
alice.grew_up()  
print(alice)          # Alice, 21 years old  
  
alice.grew_up(10)  
print(alice)          # Alice, 31 years old
```

Інкапсуляція, таким чином, зосереджує код в одному місці, не розпорошуючи його по всій програмі, що дає змогу застосовувати операції до усіх екземплярів і полегшує супровід програмного забезпечення<sup>2</sup>.

## Взаємодія з іншими екземплярами

Метод `grew_up()` використовує атрибути лише того екземпляра, для якого він викликається. Напишемо метод `age_difference()`, який використовуватиме атрибути іншого екземпляра, а саме, повертатиме різницю у віці між двома особами.

```
class Person:  
    info = "I'm a Person."  
  
    ...  
  
    def age_difference(self, other):  
        return abs(self.age - other.age)  
  
bob = Person('Bob', 25)  
print(alice.age_difference(bob))      # 6  
print(bob.age_difference(alice))      # 6
```

---

<sup>2</sup> В інших мовах програмування інкапсуляція має дещо інше значення. Вона дає змогу поділяти атрибути і методи класу на публічні, приватні та захищені. З імітацією такої інкапсуляції в Python можна ознайомитися на прикладі кодів, наведених в додатках до цієї лекції - ALL IS PUBLIC і GETTERS & SETTERS.

## Перевантаження операторів

Перевантаження операторів дає змогу екземплярам класів бути складовими виразів, тобто брати участь у різноманітних операціях (арифметичних, порівняння, доступ до елемента за індексом, зрізи та ін.). Для кожного конкретного оператора зарезервоване спеціальне ім'я методу, наприклад:

```
__add__()      # оператор +
__sub__()      # оператор -
__mul__()      # оператор *
__div__()      # оператор /
__mod__()      # оператор %
__eq__()       # оператор ==
__ne__()       # оператор !=
__lt__()       # оператор <
__le__()       # оператор <=
__gt__()       # оператор >
__ge__()       # оператор >=
__getitem__()  # оператор []
```

Операції, які здійснюються з використанням таких операторів, еквівалентні викликам відповідних методів, наприклад, `x + y` те саме, що `x.__add__(y)`.

Якщо клас не перевантажує якогось оператора, то це означає, що екземпляри класу не підтримують відповідну операцію, а спроба її виконати викличе помилку:

```
alice + 3
TypeError: unsupported operand type(s) for +: 'Person' and 'int'
```

Для прикладу перезавантажимо оператор "+" у класі `Person` так, щоб він діяв аналогічно методу `grew_up()`:

```
class Person:
    ...

    def __add__(self, x):
        self.age += x

alice = Person('Alice', 20)
print(alice)          # Alice, 20 years old
```

```
alice + 3
print(alice)          # Alice, 23 years old
```

Перевантажимо тепер оператор порівняння <:

```
class Person:

    ...

    def __lt__(self, other):
        return self.age < other.age
```

Тепер можна порівнювати та впорядковувати осіб за віком:

```
alice = Person('Alice', 20)
bob = Person('Bob', 25)
mary = Person('Mary', 15)

print(alice < bob)      # True
print(bob < mary)       # False

persons = [alice, bob, mary]
persons.sort(reverse=True)

print(*map(str, persons))
```

Але, якщо Ви перевантажили не всі оператори порівняння, слід бути обережним, бо, наприклад:

```
print(bob > mary)      # True
```

але

```
print(bob >= mary)
TypeError: '>=' not supported between instances of 'Person'
and 'Person'
```

## ДОДАТКИ:

### ALL IS PUBLIC

# NO REALLY ENCAPSULATION IN PYTHON - ALL IS PUBLIC!

```
class Person:

    def __init__(self, name, age, gender):
        self.name = name          # public
        self._age = age           # declaratively private
        self.__gender = gender    # pseudo private

alice = Person('Alice', 20, 'female')

print(alice.name)                # Alice

print(alice.age)
AttributeError: 'Person' object has no attribute 'age'
print(alice._age)                # 20

print(alice.gender)
AttributeError: 'Person' object has no attribute 'gender'
print(alice.__gender)
AttributeError: 'Person' object has no attribute '__gender'
print(alice._Person__gender)     # female
```

### GETTERS & SETTERS

```
class Person:

    def __init__(self, name, age, gender):
        self.name = name
        self._age = age

    # getter (way 1)
    def get_age(self):
        return self._age

    # getter (way 2)
    @property
    def age(self):
```

```
        return self._age

    # setter (way 1)
    def set_age(self, value):
        self._age = value

    # setter (way 2)
    @age.setter
    def age(self, value):
        self._x = value

alice = Person('Alice', 20)

# getter (way 1)
print(alice.get_age())      # 20

# getter (way 2)
print(alice.age)            # 20

# setter (way 1)
alice.set_age(25)
print(alice.age)            # 25

# setter (way 2)
alice.age = 30
print(alice.age)            # 30
```